PES University — Department of Computer Science & Engineering

Lab 1: Requirements Engineering & UML Use-Case Modelling

Problem Statement #42: **Internal Microservice Catalog & Health Portal** | Student: **Subramani B M** | SRN: **PES1UG24CS473** | Section: **H**

1. Functional Requirements (FR-001 to FR-005)

ID	Type	Description	Priority	Acceptance Criteria	Rationale
FR-001	Health Monitoring	The system shall ping microservice health endpoints at 30-second intervals and compute rolling 24-hour service availability percentages.	High	**Pass:** Downtime recorded and alert dispatched on 3 consecutive failed health probes. **Fail:** Unreachable service marked healthy.	Continuous health monitoring is essential for ensuring service reliability and enabling rapid incident response in a microservice architecture.
FR-002	Catalog Management	The system shall maintain a searchable catalog of all registered microservices, storing metadata including service name, owner team, version, repository URL, deployed environment(s), and API documentation link.	High	**Pass:** A newly registered microservice appears in search results within 60 seconds and all metadata fields are accurately displayed. **Fail:** A registered service is missing from the catalog or displays stale/incorrect metadata.	A centralized catalog gives DevOps engineers and system architects a single source of truth for discovering and understanding available microservices.
FR-003	Dependency Mapping	The system shall automatically discover and render an interactive dependency graph showing upstream and downstream relationships among all registered microservices.	High	**Pass:** Adding a dependency between two services is reflected in the graph within 2 minutes; clicking a node navigates to the service detail page. **Fail:** A known dependency is absent from the graph, or the graph displays a dependency that does not exist.	Visualizing inter-service dependencies helps system architects identify critical paths, single points of failure, and the blast radius of potential outages.
FR-004	Alerting & Notifications	The system shall dispatch real-time downtime alert notifications (via email, Slack, and webhook) to the designated on-call contacts of a service when the health checker detects an outage (3 consecutive failed probes).	Medium	**Pass:** On-call contacts receive a notification on all configured channels within 60 seconds of the third consecutive failed probe. **Fail:** An outage occurs and no notification is sent, or the notification is sent to the wrong contact.	Timely, multi-channel alerting ensures the right personnel are informed immediately, minimizing mean time to acknowledge (MTTA) and mean time to resolve (MTTR).
FR-005	API Documentation	The system shall aggregate and display auto-generated API documentation (OpenAPI / Swagger specs) for each microservice, enabling developers to browse endpoints, request/response schemas, and example payloads.	Medium	**Pass:** Uploading or linking an OpenAPI spec renders a browsable, accurate API reference page for the service within 2 minutes. **Fail:** The rendered documentation is missing endpoints, shows incorrect schemas, or fails to render entirely.	Centralized API documentation reduces onboarding time for new developers and prevents integration errors caused by consulting outdated or scattered documentation.

2. Non-Functional Requirements (NFR-001 & NFR-002)

ID	Type	Description	Priority	Acceptance Criteria	Rationale
NFR-001	Performance & Scalability	The service catalog dependency graph viewer must render interactive node architectures containing up to 200 services smoothly with pan, zoom, and click interactions completing within 2 seconds on a standard desktop browser.	High	**Pass:** Benchmarking tests confirm graph render time ≤ 2 seconds and interaction latency ≤ 200 ms under simulated peak load (50 concurrent users, 200-node graph). **Fail:** Render time exceeds 2 seconds or UI becomes unresponsive during interaction.	A sluggish dependency graph undermines its usefulness; architects need fluid interaction to trace complex dependency chains during incident analysis and capacity planning.
NFR-002	Security & Compliance	All communication between the portal and microservice health endpoints shall be encrypted via TLS 1.2 or higher, and access to the portal shall require authentication via enterprise SSO (OAuth 2.0 / SAML). Health check credentials must be stored in an encrypted secrets vault.	High	**Pass:** Penetration testing confirms no plaintext credential exposure; unauthorized users are denied access; all health-check traffic uses TLS 1.2+. **Fail:** Any credential is transmitted or stored in plaintext, or an unauthenticated user gains access to the portal.	The portal aggregates sensitive infrastructure data (service topology, health status, API specs). Unauthorized access could expose internal architecture to attackers, and unencrypted health probes could leak credentials.